

VeriContest's ProofGen Benchmark Is Inaccurate

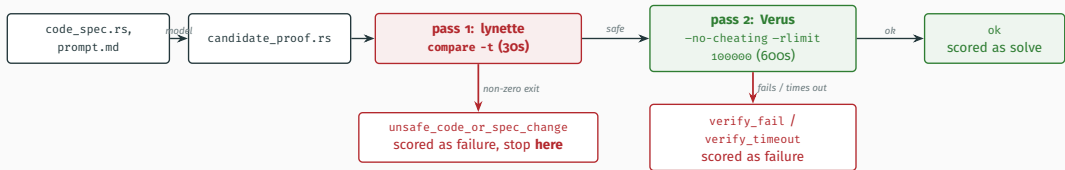
Its anti-cheat filter, lynette, is the wrong check for the job

August 25, 2026

UIUC SRSE 2026 · Mahadi

How ProofGen scores a candidate: two passes

A candidate's proof must clear **two** separate checks, in order, before it counts as solved.



Only candidates lynette marks `safe` ever reach Verus. Everything this deck asks is about that first pass.

Why a gate is needed at all

ProofGen hands the model one file to edit, and grades it with Verus. But Verus checks a proof against whatever spec and code that file *contains*, and the model wrote the whole file.

What Verus checks, vs. what the task asks

Verus checks: $\text{verifies}(S_{\text{cand}}, C_{\text{cand}}, P_{\text{cand}}) \in \{0, 1\}$: a claim about whatever file the model wrote.

the task asks: $\text{solve}(S_{\text{gt}}, C_{\text{gt}})$: a claim about the frozen problem it was actually given.

These coincide only if $S_{\text{cand}} = S_{\text{gt}}$ and $C_{\text{cand}} = C_{\text{gt}}$.

ground-truth obligation

ensures

`result == 2 * x,`

a weakened spec, trivially provable

ensures

`true,`

Absent a filter, a model facing an unprovable obligation could weaken the spec instead of finding the proof, and Verus would happily verify a different, easier problem it was never asked to solve. The gate's job is to make "the spec is still the ground-truth spec, and the code is still the given code" true *before* Verus's verdict is treated as an answer to the actual task.

How this started

I was testing ideas against the baseline **proof-repair** loop. The results would not come out consistent. That sent me into the ProofGen scoring pipeline, where a filter called **lynette** decides whether a candidate reaches Verus at all.

gpt55, ProofGen so, 946 problems

	Verus ok	Verus fail	total
lynette safe	132	118	250
lynette unsafe	49	647	696
total	181	765	946

Pipeline reports **132 / 946 = 13.95%**

Verus alone accepts **181 / 946 = 19.13%**

The gate is the entire difference: +5.18pp, +37.1% relative.

Invisible by construction

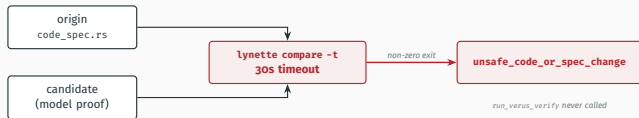
The harness runs Verus *only* on candidates the gate already passed, so “gate-rejected but Verus-accepts” cannot appear in any recorded summary JSON. It became visible only by re-running Verus on all 946 with the gate bypassed.

Not cheating by another route

These 49 verify under `-no-cheating`, which rejects `assume`, `admit`, and `external_body`. Assume-smuggling is a different cell: 57 of the 118 that pass the gate and then fail Verus.

cf148A: how lynette gates a candidate

Codeforces cf148A, gpt55 so. Smallest of the 14 if-drop cases (47-line code_spec.rs); the model's entire addition is one if/else. Its own gold verified.rs **passes** this same gate, so this is not a problem the gate can never satisfy: it is wrong about *this candidate*.



The call

test_spec_code2proof.py:108 calls
code_change_is_safe in target mode: exactly the
gate above. Only candidates marked safe ever
reach run_verus_verify.

count = count + 1; and i = i + 1; are
byte-identical to the original.

what the model added: the else branch

```
} else {  
    assert(!damaged(i as int, k as int,  
        l as int, m as int, n as int));  
    assert(count_damaged_spec(k as int,  
        l as int, m as int, n as int, i as int)  
        == count_damaged_spec(k as int,  
        l as int, m as int, n as int, i as int - 1));  
}
```

What lynette actually compares

Deghosted view of both sides, from `lynette compare -t -v`:

code_spec.rs, as lynette sees it

```
while i <= d {  
  if i % k == 0 || i % l == 0  
    || i % m == 0 || i % n == 0 {  
    count = count + 1;  
  }  
  i = i + 1;  
}
```

Why it rejects

`deghost.rs:125-147`, the `Expr::If` arm: an all-assert else deghosts to `None`; `None.map(...)` is `None`; the entire `ExprIf` node is dropped, then-branch included. The counting logic disappears from the candidate's view, and the gate reports a difference it created itself.

candidate, as lynette sees it

```
while i <= d {  
  i = i + 1;  
}
```

Why it should have been accepted. Verus verifies the candidate in 0.4s under `-no-cheating`, and the gold verified.rs for this problem is safe. A complete, correct proof is recorded as tampering because the tool deleted real code before comparing it.

44 of the 49 should be accepted

gpt55, so, pin fce8da7. Each of the 49 re-cut by *cause*:

cause	n	call
lynette if-drop artifact	20	accept
subexpression / return-value hoisting	11	accept
inert let old_x = x; insertions	8	accept
contract added to a previously-bare helper	3	accept
added use import only	1	accept
type annotation only	1	accept
for → while rewrite	3	reject
new exec functions added	1	reject
genuine semantic edit	1	reject

Corrected: 176 / 946 = 18.60% (+4.65pp, +33.3% relative)

repair@k oscillates: $A \rightarrow B \rightarrow A$

pass@k drops a rejected sample. repair@k reverts.

A gate rejection does not remove one candidate out of k : it resets the trajectory to the last accepted state and tries again, so the sequence of states bounces **A** (safe) $\rightarrow$ **B** (attempt) $\rightarrow$ **A** (reverted), instead of moving forward.

on rejection: the repair loop's revert

```
state.current_program = state.last_safe_program
state.last_error       = CODE_SPEC_CHANGED_ERROR
state.solved_round     = max(0, round_index - 1)
```

current_program reverts to the last safe program: what Verus actually receives next round.

last_error is set to CODE_SPEC_CHANGED_ERROR: what the next prompt is told, whether or not the model actually changed anything.

solved_round rolls back too, if that reverted program itself verifies.

One rejection costs pass@k one sample out of k . In repair@k it costs the *chain*: progress is discarded, and the next round starts from the accusation, not from where the model actually was.

Ground truth fails the gate 31.8% of the time

Every problem ships a `verified.rs`: the benchmark's own reference proof, a known-correct answer to the exact task the model is given. Run the gate against it directly.

```
lynette compare -t code_spec.rs VS verified.rs (n = 946, pin fce8da7)
```

	count	share
accept	645	68.2%
REJECT	301	31.8%

A third of the benchmark's own correct answers are classified as having tweaked the spec or the code. The reference proofs are genuine: only one of the 946 fails Verus.

cf1200C: the entire objection is a library import

The reference proof needs a `vstd` lemma, so it imports it. Its complete deghosted difference from `code_spec.rs`:

cf1200C: `verified.rs` vs `code_spec.rs`, deghosted

```
use vstd::prelude::*;  
+ use vstd::arithmetic::div_mod::*;
```

That is the whole diff, and the gate exits 1. `use` items pass through erasure untouched, so any proof needing a `vstd` import beyond what `code_spec.rs` already has can **never** pass, however clean the proof.

52 / 52

of `verified.rs` files that add a `use` line are rejected. Of the 645 the gate accepts, 0 add one.

lc2094: code_spec.rs was never fully built

The rejection here is not about the proof: `code_spec.rs` is missing contracts its own reference proof needs, so `verified.rs` supplies them and is charged with tampering for it. Helper `count_digit_exec`:

file	what it says about <code>count_digit_exec</code>
<code>code.rs</code>	bare: <code>fn count_digit_exec(...) -> i32 {</code>
<code>spec.rs</code>	<code>absent: states only the target's contract and spec fns</code>
<code>code_spec.rs</code>	an ensures, but no requires
<code>verified.rs</code>	the requires, plus a second ensures clause

the gate's diff: `verified.rs` filling in what `code_spec.rs` left out

```
fn count_digit_exec(nums: &Vec<i32>, digit: i32) -> i32

+ requires nums.len() <= 100,

+ forall|i: int| 0<=i<nums.len() ==> 0<=#[trigger] nums[i]<=9,

  ensures c as int == Self::count_digit(nums@, digit as int, nums.len() as int),

+ 0 <= c <= nums.len(),
```

Why this happens, and why nothing caught it

Why this happens

`code_spec.rs` is not assembled from `code.rs` + `spec.rs`. The benchmark's authoring skill builds it by *back-propagating* from `verified.rs` after the proof succeeds: "update `code_spec.rs` to match the verified algorithm structure (without proof, ghost vars, loop invariants, and decreases)." Contracts are not on that removal list, which is why `code_spec.rs` carries helper contracts absent from both `code.rs` and `spec.rs`. That step is manual, and here it was done incompletely.

Why nothing caught it

The benchmark's consistency checker asserts

$$\text{code_spec.rs} \subseteq \text{verified.rs}$$

an ordered line-subsequence, one-directional. A contract the author forgot to back-propagate makes `code_spec.rs` *smaller*, which still satisfies $\subseteq$. lc2094 passes the benchmark's own QA cleanly.

The model is handed an under-specified file, and the frozen "ground truth" it is measured against is a partially back-propagated proof artifact, not the stated specification in `spec.rs`.

Five ways the gate manufactures tampering (1/2)

The gate tests **equality** of the proof-erased remainder. ProofGen needs **extension**: output = original + proof-only additions. Equality cannot express “additions are allowed,” and the erasure itself is buggy. Every category below follows from that.

1. a ghost-only else deletes the whole if

```
if a[i] > t {  
  cnt = cnt + 1;  
+ } else {  
+   proof { assert(cnt as int  
+     == count_prefix(a@, i+1)); }  
}
```

Lynette then sees nothing: the whole statement, `cnt = cnt + 1` included, is gone. A comment-only `else` does it too.

2. an added use import

```
+ use vstd::arithmetic::div_mod::*;
```

use items pass through erasure untouched (`cf1200C`, `Q5`).

4. a ghost witness written `let`, not `let ghost`

```
+ let old_best = best;  
    // read only inside asserts
```

Five ways the gate manufactures tampering (2/2)

3. invariant/decreases on a bare loop

```
loop
+ invariant i <= n,
+ decreases n - i,
{
```

Verus cannot verify a bare loop without them: no correct proof of such a problem can pass.

5. a contract on a helper that shipped bare

```
- fn helper(v: &Vec<i32>) -> usize {
+ fn helper(v: &Vec<i32>) -> usize
+ ensures result <= v.len(),
+ {
```

Without it, the caller cannot be proven at all.

The relation we need, and the one we have

The gate's job is **task identity**: is the candidate still an answer to the problem that was handed out?

Want vs. have

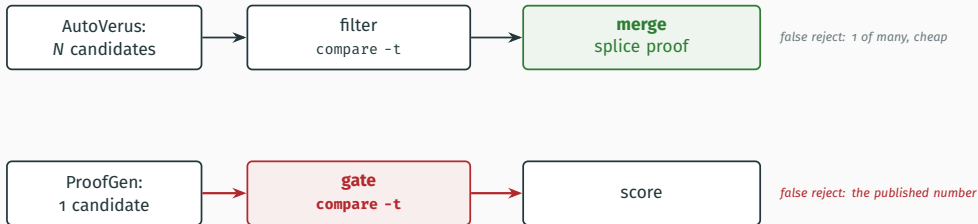
want: $\text{accept}(O, F) \iff F = O + \Delta, \text{ every } \delta \in \Delta \text{ proof-only}$ (*extension*)

have: $\text{accept}(O, F) \iff \text{erase}(O) = \text{erase}(F)$ (*equality*)

Equality is an **equivalence relation**: symmetric. Extension is a **partial order**: antisymmetric. “Additions are allowed, in one direction only” is not expressible in a symmetric relation. That is the entire defect, stated once. Every category in Q6 is an addition that an equality test structurally cannot represent.

Why lynette is built that way

lynette is AutoVerus's tool, and `compare` is literally the precondition its **merge** asserts (`fmerge_files`). To splice proof content from one candidate into another you must know they are the *same program modulo proof*: for merging, equality is exactly the right relation. VeriContest imported that check through `evaluation.util.code_change_is_safe` and repurposed it as a cheating detector.



The cost asymmetry flips on import. AutoVerus filters N candidates: a false reject is a rational trade. ProofGen scores one candidate per problem: the same false reject is measurement error in a published number. Same behaviour, opposite economics. The tool is not malfunctioning: it is being used outside its design contract.

The benchmark already knows the right relation

VeriContest's own authoring checker asserts:

extension, already, in the QA

`code_spec.rs` $\subseteq$ `verified.rs` (ordered subsequence, i.e. extension)

check	pass rate
authoring checker ($\subseteq$, extension)	946 / 946
scoring pipeline (compare -t, equality)	645 / 946

The benchmark defines the task as extension in its QA, and scores it as equality in its pipeline.

So what do we do? Three paths

1. Drop the gate.

Trust the model not to touch spec or code, and spend the effort on repair@k instead.

2. Build our own AST-based gate.

vgate, in tools/vgate/, with minimal engineering. It also identifies where `code_spec.rs` needs patching against `verified.rs`.

3. Treat the filter as a general problem.

Given a spec S , code C , and an arbitrary file Z , decide by AST whether Z has the same spec as S and the same code as C .

Where each lands

gate	GT accepted	gpt55 pass@1
lynette compare -t (current)	68.2%	13.95%
vgate (prototype)	83.4%	15.12%
no gate at all	99.9%	19.13%

vgate is a prototype: validated on the 946 reference proofs and on gpt55 so only, never replayed against the recorded candidate runs, with an unresolved `let` vs `let` ghost policy and four known gaps in its README.